SAGA Pattern (Choreography-Based) – Real-Life Analogy & Easy Example

Simple Definition

Saga Pattern is used when a business process (like placing an order) involves multiple microservices, and we want to make sure each service completes its task. If anything fails, we roll back the changes using compensating actions.

Think of it like ordering food online – if payment fails, the kitchen shouldn't cook your food.

Simple Real-World Example:

Imagine this flow:

1. **User places an order for a product.**
2. **Order Service receives the request → sends an order event to Kafka.**
3. **Payment Service listens to the order event → deducts money → sends payment status back via Kafka.**
4. **Order Service again listens to payment status → updates order status based on success or failure.**

Why We Need This (The Problem)

Without Saga:

- **If Payment fails after Order is created, we have to manually clean up.**
- **No coordination between services → leads to data inconsistency.**

With Saga:

- **Each service works independently.**
- **They listen to events and take actions.**
- **On failure, previous steps can be undone automatically.**

Flow from the Image (Simplified)

Services Involved:

- **Order Service**
- **Payment Service**
- **Kafka Topics: order-event, payment-event**

Step-by-Step:

1. **Order Service → sends this to order-event:**

{"userId":101, "productId":3547, "price":1200}

2. **Payment Service → listens to it, checks if the user has enough balance:**

- **If yes: deducts amount, and sends success message to payment-event**
- **If no: sends failed message**

3. **Order Service again listens to payment-event:**

- **If "SUCCESS" → sets orderStatus to "COMPLETED"**
- **If "FAILED" → sets orderStatus to "CANCELLED"**

Diagram Breakdown (Easy Labels)

User → Order Service → Kafka (order-event) → Payment Service ↓ Kafka (payment-event) ↓ Order Service updates status Sample Event Messages:

1. **Sent by Order:**

{"userId":101, "productId":3923, "price":3000}

2. **Payment replies:**

{"userId":101, "productId":3923, "price":3000, "paymentStatus":"SUCCESS"}

How It Solves the Problem

Without Saga	With Saga (Event-based)
Hard to track payment failure	Payment failure is reported via event
Manual rollback needed	Compensating actions handled automatically
Services tightly coupled	Loosely coupled via Kafka
Absolutely! Let's break down Orchestration-based SAGA in the same simple style	
as before — using a real-world analogy, easy-to-follow flow, and beginner-friendly	
concepts.	

Saga Pattern (Orchestration-Based) – Easy Notes with Analogy

Simple Definition

In Orchestration-based Saga, there is a central controller (called a Saga Orchestrator) that manages the flow of all microservices involved in a transaction. The orchestrator tells each service what to do next, and what to do if something fails.

Think of it like a wedding planner (orchestrator) who coordinates caterer, venue, photographer. If the caterer cancels, the planner takes care of rescheduling or canceling the whole event.

Real-World Inspired Microservices Example

Let’s say a user places an order. The process involves:

1. **Order Service**
2. **Payment Service**

3. **Inventory Service**
4. **Shipping Service**

Step-by-Step Flow

User → Order Service → Orchestrator → Payment → Inventory → Shipping

↘ ↘

Refund Release Stock (on failure)

Why We Need Orchestration (The Problem It Solves)

Without Saga:

- **No clear control over transaction flow.**
- **Services might not agree on who failed and what to undo.**
- **Logic becomes scattered across services (like in Choreography).**

With Orchestration:

- **One place controls the flow**
- **Compensating actions are pre-defined.**
- **Easier to debug, maintain, and monitor.**

What is the Orchestrator?

A Spring Boot service that:

- **Starts the saga by calling Order Service.**
- **Then calls Payment Service.**
- **If success → calls Inventory Service.**
- **If any step fails, it:**
- **Cancels previous steps (e.g., refunds money).**
- **Updates the order as failed.**

Flow with Simple Example

Step 1: Orchestrator starts

{ "userId": 101, "productId": 3923, "price": 3000 }

Step 2: Calls Payment Service

- **If Payment succeeds → Call Inventory**
- **If Payment fails → Mark order as failed**

Step 3: Calls Inventory Service

- **If Inventory is available → Proceed to shipping**
- **If not → Refund payment, cancel order**

Visual Summary

+-------------------------+

| Saga Orchestrator |

+-----------+-------------+

|

v

Order Service

|

v

Payment Service or

|

v

Inventory Service or

|

v

Shipping Service or

Compensation Example

- **If Inventory fails, Orchestrator calls:**
- **paymentService.refund()**
- **orderService.cancelOrder()**

Summary Table

Concept	Meaning
Orchestrator	Central controller for all steps
Local transactions	Each service runs its own transaction
Compensation	Rollback previous step if failure occurs
Simpler monitoring	Easier to observe and debug
Code Complexity	More centralized (but sometimes tightly coupled)

Spring Boot Example (High-Level)

OrderSagaOrchestrator.java

```
public class OrderSagaOrchestrator {
 public void start(OrderRequest request) {
 orderService.create(request);
 boolean paid = paymentService.pay(request);

 if (!paid) {
```

```
orderService.cancel(request);
return;
}
boolean reserved = inventoryService.reserve(request);
if (!reserved) {
paymentService.refund(request);
orderService.cancel(request);
return;
}
shippingService.ship(request);
orderService.complete(request);
}
}
```

This is procedural, but in real-world projects you can use Temporal, Camunda, or a workflow engine for complex sagas.

Easy Line to Remember

"In orchestration-based saga, one smart boss (orchestrator) gives tasks to others, checks their work, and handles cleanup if anyone messes up."

Great choice! Let's break down CQRS (Command Query Responsibility Segregation) in the same easy-to-understand format with real-world analogies, diagrams, problems it solves, and a simple Spring Boot example.

CQRS (Command Query Responsibility Segregation) – Easy Notes

Simple Definition

CQRS is a pattern where you separate the logic for:

- **Command → operations that change data (Create, Update, Delete)**
- **Query → operations that read data (GET) Instead of using one model (class, service, or database table) for both reading and writing, CQRS splits them for better scalability and performance.**

"Use one model to write data, and a separate optimized model to read it."

Real-World Analogy

Imagine an e-commerce store:

- **A warehouse worker updates the inventory stock (Command).**
- **A customer searches for products (Query). They have different goals and different ways of interacting with the system. You don't want to slow down search just because stock updates are happening.**

Why Do We Need CQRS? (The Problem It Solves)

Without CQRS:

- **Same model handles both read and write.**
- **Hard to optimize for both: reads need performance; writes need integrity.**
- **Changes in one use case may affect the other.**

With CQRS:

- **Read model can be denormalized, cached, indexed for fast access.**
- **Write model can be transactional and secure.**
- **Helps in event-driven architecture and scaling systems.**

Event-Driven Flow: Why & How

Why:

- **In CQRS, the read DB is separate from the write DB.**
- **If we only update the write DB, the read side will show stale data.**
- **So we need to inform the read side when something changes.**

How:

- **After every write (create/update/delete), the Command Service publishes an event.**
- **The Read Service listens to the event, extracts the data, and updates its own read-optimized database.**

Real-Life Analogy (Very Easy)

Imagine a chef (Command side) making food. Once done, they ring the bell (event) to let the waiter (Read side) know that food is ready and can be served. Chef ≈ Write Bell ≈ Event Waiter ≈ Read

Real CQRS + Event-Driven Flow

Client

|

v

Command API ---> Write DB

|

+--- Publish "OrderCreated" Event ---> Kafka/RabbitMQ

|

v

Read Service (Consumer)

|

v

Update Read DB

Spring Boot Example with Event Publishing

After Creating Order in Command Handler

```
public void handle(CreateOrderCommand cmd) {
```

```
 Order order = new Order(cmd.getProductId(), cmd.getQuantity());
 orderRepository.save(order);
 // Publish Event
 OrderCreatedEvent event = new OrderCreatedEvent(order.getId(),
order.getProductId(),
order.getQuantity());
 kafkaTemplate.send("order-events", event);
}
```

Read Service (Event Listener)

```
@KafkaListener(topics = "order-events", groupId = "read-db-updater")
public void consume(OrderCreatedEvent event) {
 // Save to Read DB (NoSQL, cache, etc.)
 ReadOrder readOrder = new ReadOrder(event.getOrderId(), event.getProductId(),
event.getQuantity());
 readOrderRepository.save(readOrder);
}
```

What Kind of Events Can Be Published?

Action	Event Example
Create Order	OrderCreatedEvent
Update Product	ProductUpdatedEvent
Delete User	UserDeletedEvent

Action	Event Example
Each event carries the necessary data payload to keep the read side updated.	

Summary Table (Now Complete!)

CQRS Concept	With Event-Driven Support
Command Side	Handles writes, publishes domain events
Query Side	Handles reads, updates based on those events
Communication	Asynchronous via Kafka, RabbitMQ, etc.
Read DB	Fast, denormalized, possibly NoSQL
Benefit	Loose coupling, high scalability, real-time UI

One-Liner to Remember

"CQRS writes data and tells others via events — the read side listens and updates itself." Excellent! The Transactional Outbox Pattern is a powerful technique used in eventdriven microservices to ensure data consistency between the database and the event system (like Kafka or RabbitMQ). Let's break it down in the same simple-to-remember format you've been using.

Transactional Outbox Pattern – Easy Notes with Analogy & Example

---

Simple Definition

The Transactional Outbox Pattern solves the problem of reliably publishing events when data is updated in a database. Instead of publishing the event directly (which can fail), the event is first stored in a special "outbox" table inside the same

database transaction, and later a background process reads from the outbox and publishes it to the message broker.

Problem It Solves

Imagine you save an order to the database and then try to publish an OrderCreatedEvent to Kafka.

Without this pattern:

- **What if DB save is successful, but Kafka publish fails? → Your system becomes inconsistent. Kafka never knows the order was created.**

With this pattern:

- **You save order data and event data (in outbox) in the same transaction.**
- **So either both succeed or both fail.**
- **Later, a reliable background job picks up the event from the outbox and sends it to Kafka.**

Real-Life Analogy

Think of a train ticket counter:

- **They write your booking details and a receipt in the same file.**
- **Later, a clerk sends out those receipts to the customer inbox.**
- **Even if power goes out after saving, the receipt is still on record and will be sent later.**

Simple Flow Diagram

Client

|

v

+------------------+

| Order Service |

| |

| - Save order |

| - Save event |

| to outbox |

+------------------+

|

[DB Transaction]

|

v

Outbox Table <-- Polling Service --> Kafka

Spring Boot Example – High-Level Flow

1. Order Save + Event Save (in DB transaction)

```
@Transactional
public void createOrder(OrderRequest request) {
 Order order = new Order(request.getProductId(), request.getQuantity());
 orderRepository.save(order);
 OutboxEvent event = new OutboxEvent("OrderCreated", toJson(order));
 outboxRepository.save(event);
}
```

---

2. Polling Service to Send Events

```
@Scheduled(fixedRate = 5000)
public void publishOutboxEvents() {
 List<OutboxEvent> events = outboxRepository.findUnpublished();
 for (OutboxEvent event : events) {
```

```
 kafkaTemplate.send("order-events", event.getPayload());
 event.markAsPublished(); // or delete after success
 }
 outboxRepository.saveAll(events);
}
```

Outbox Table – Looks Like This

ID	EventType	Payload (JSON)	Published
1	OrderCreated	{ "orderId": 123, "amount": 1000 }	false
2	PaymentDone	{ "paymentId": 789, "status": "OK"}	true

Benefits of Transactional Outbox

Problem	Solution with Outbox
Event lost if broker fails	No — event is stored in DB first
Partial failure (DB ok, event lost)	Solved — single DB transaction ensures consistency
Difficult to debug or retry	Easy — outbox table keeps a record

---

Tools That Help

- **Debezium: Automatically captures outbox table changes and sends to Kafka (CDCbased outbox)**
- **Transactional Outbox Libraries: e.g., [Axon Framework], [Eventuate]**
- **Manual Polling: As shown in example above**

One-Liner to Remember

"Transactional Outbox stores the event in the DB first — publish later, safely and reliably."

Aggregator Design Pattern

Simple Definition

The Aggregator Pattern is used when a single request from the client needs data from multiple microservices. Instead of calling all services from the client, an Aggregator service collects responses from different services and returns a combined result.

Think of it like a restaurant waiter taking one order and coordinating with multiple kitchens (Chinese, Indian, Italian) to serve a full combo meal.

Problem It Solves

Without Aggregator:

- **The client has to call multiple microservices.**
- **Client becomes aware of internal service URLs and logic.**
- **Error handling, retries, timeouts become the client's responsibility.**

With Aggregator:

- **The Aggregator handles all service calls.**
- **Client gets one simple and unified response.**
- **Helps in API composition, orchestration, and performance optimization.**

Real-World Example: E-commerce Order Summary

Services Involved:

1. **Order Service → gives order details**
2. **Payment Service → payment status**

3. **Shipping Service → delivery status**

Client wants:

“Give me full order summary.”

Aggregator Flow Diagram

+----------+

Client → | Aggregator |

+----------+

| | |

↓ ↓ ↓

Order Payment Shipping

Service Service Service

→ Combine Responses →

↑

Unified JSON

Simple JSON Response from Aggregator

```
{
 "orderId": "123",
 "items": ["Shirt", "Shoes"],
 "paymentStatus": "PAID",
 "deliveryStatus": "Shipped"
}
```

Spring Boot Example – Aggregator Controller

1. Aggregator Service (Java)

```
@Service
```

```
public class OrderSummaryAggregator {
 @Autowired RestTemplate restTemplate;
 public OrderSummary getOrderSummary(String orderId) {
 Order order = restTemplate.getForObject("http://order-service/orders/" + orderId,
Order.class);
 Payment payment = restTemplate.getForObject("http://payment-service/payments/"
+
orderId, Payment.class);
 Shipping shipping = restTemplate.getForObject("http://shipping-service/shipments/"
+
orderId, Shipping.class);
 return new OrderSummary(order, payment, shipping);
 }
}
```

2. Controller Layer

```
@RestController
@RequestMapping("/summary")
public class OrderSummaryController {
 @Autowired OrderSummaryAggregator aggregator;
 @GetMapping("/{orderId}")
 public OrderSummary get(@PathVariable String orderId) {
 return aggregator.getOrderSummary(orderId);
 }
}
```

When to Use Aggregator Pattern

Use Case	Pattern Benefit
Dashboard showing multiple sources	Combine service calls into one response
Composite API endpoints	Reduce number of calls for mobile/web clients
Pre-processing data for front-end	Format and enrich data before sending

---

Bonus Tip: Use with API Gateway

You can also place the Aggregator behind an API Gateway, so the gateway has multiple responsibilities:

- **Simple routing → handled by gateway**
- **Complex composition → handled by Aggregator microservice**

One-Liner to Remember

“Aggregator Pattern lets one microservice fetch and combine data from many others, so clients don’t have to.”

Event Sourcing in Microservices – Easy Notes

Simple Definition

Event Sourcing is a pattern where instead of saving the final state of data (like a row in a database), you store every change (event) that happens to that data. The current state is then rebuilt by replaying all events.

Think of it like a bank passbook: You don’t just store the current balance — you store every deposit and withdrawal. You can recalculate the balance anytime by replaying all transactions.

Problem It Solves

Without Event Sourcing	With Event Sourcing
You only see the final state	You can see how it got there
Data loss if DB update fails midway	Each step is stored safely as an event
No audit/history trail	Complete history is available

Real-World Use Case

Let's say we are building an Order Service:

1. **User creates an order → event: OrderCreated**
2. **User pays → event: PaymentDone**
3. **Order shipped → event: OrderShipped**
4. **Order cancelled → event: OrderCancelled Instead of updating a row in DB each time, we save:**

[

{ type: "OrderCreated", data: {...} },

{ type: "PaymentDone", data: {...} },

{ type: "OrderShipped", data: {...} }

]

We can replay these events to build current state: status = Shipped

Simple Diagram – Event Sourcing Flow

Client → Command API → Event Store (DB)

|

↓

Rebuild State by Replaying Events

↓

Show Final State (Query)

The event store becomes the source of truth.

Example Event History (Order)

Event Type	Data (Example)
OrderCreated	orderId=1, product="Shoes"
PaymentDone	orderId=1, amount=1200
OrderShipped	orderId=1, tracking="AB1234"

Spring Boot – Basic Event Sourcing Components

1. Save Event to DB (Event Store)

```
public class Event {
 private String id;
 private String aggregateId; // like orderId
 private String type; // e.g., OrderCreated
 private String payload; // JSON data
 private Instant timestamp;
}
public void saveEvent(String type, String aggregateId, Object data) {
 Event event = new Event();
 event.setType(type);
 event.setAggregateId(aggregateId);
 event.setPayload(toJson(data));
 event.setTimestamp(Instant.now());
 eventRepository.save(event);
```

```
}
```

---

2. Rebuild State from Events

```
public OrderState rebuildState(String orderId) {
 List<Event> events = eventRepository.findByAggregateId(orderId);
 OrderState state = new OrderState();
 for (Event e : events) {
 state.applyEvent(e); // update state based on event type
 }
 return state;
}
```

When to Use Event Sourcing

Ideal For	Reason
Financial Systems	Complete audit trail
Order Processing / E-commerce	Easy to trace status changes
CQRS Architecture	Events can feed the read model
Undo/Redo/Replay functionality	Rebuild past states

Event Sourcing vs Transactional Outbox

Concept	Focus
Event Sourcing	Save each change as an event

Concept	Focus
Transactional Outbox	Save event after state is saved

You can also combine them: use event sourcing for core domain, and use the outbox pattern to send events to Kafka.

One-Liner to Remember

“Event Sourcing stores every change as an event — so the full history builds the present.”

Hexagonal Architecture (Ports & Adapters) – Easy Notes

Simple Definition

Hexagonal Architecture is a software design pattern where the core logic (business rules) is completely isolated from external systems like databases, APIs, UIs, or message brokers. It uses:

- **Ports → interfaces that define what the core expects or provides**
- **Adapters → classes that plug into these ports (like DB, REST, Kafka)**

"It keeps your app’s brain (business logic) safe from the outside world."

Real-Life Analogy

Think of a power socket (port) at home. You can plug in:

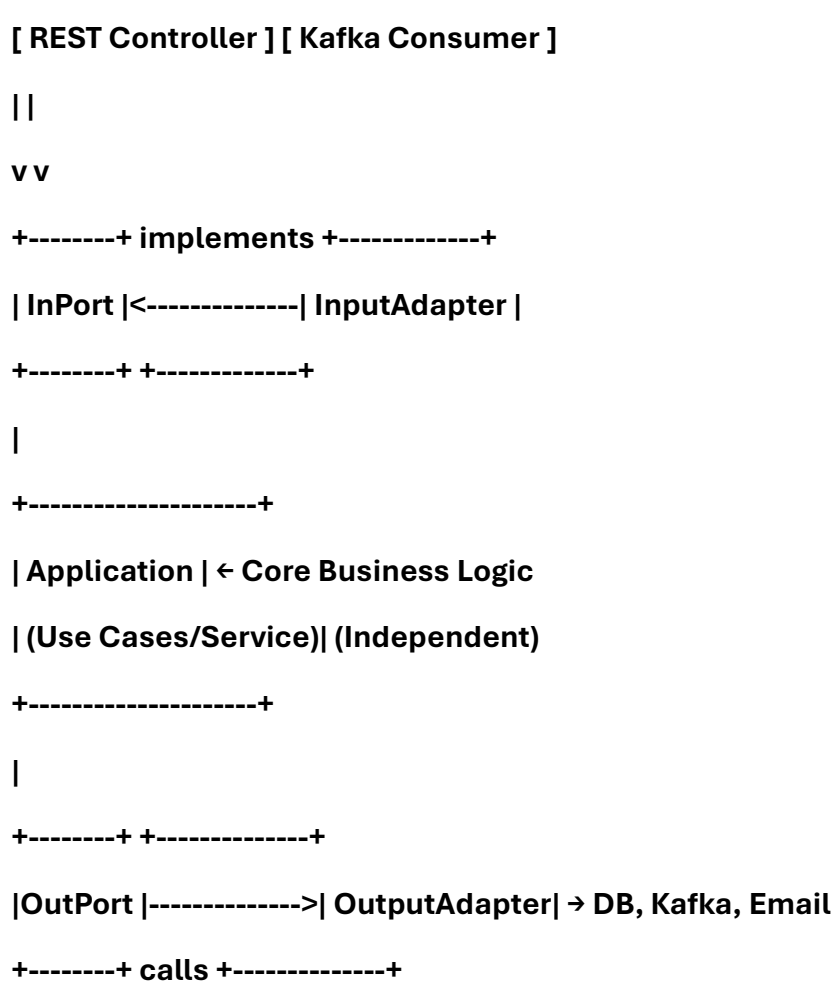
- **a phone charger**
- **a fridge**
- **a fan Each of them is an adapter. The socket doesn’t care what you plug in — as long as the plug fits!**

Why Use It? (Problems It Solves)

Problem Without It	Solved by Hexagonal Arch
Business logic tightly coupled to DB, REST, Kafka	Clean separation of concerns
Hard to test logic without real DB/API	Easily test core with mock ports
Code changes ripple everywhere	You can swap adapters without touching core

---

Diagram – Simple Visual

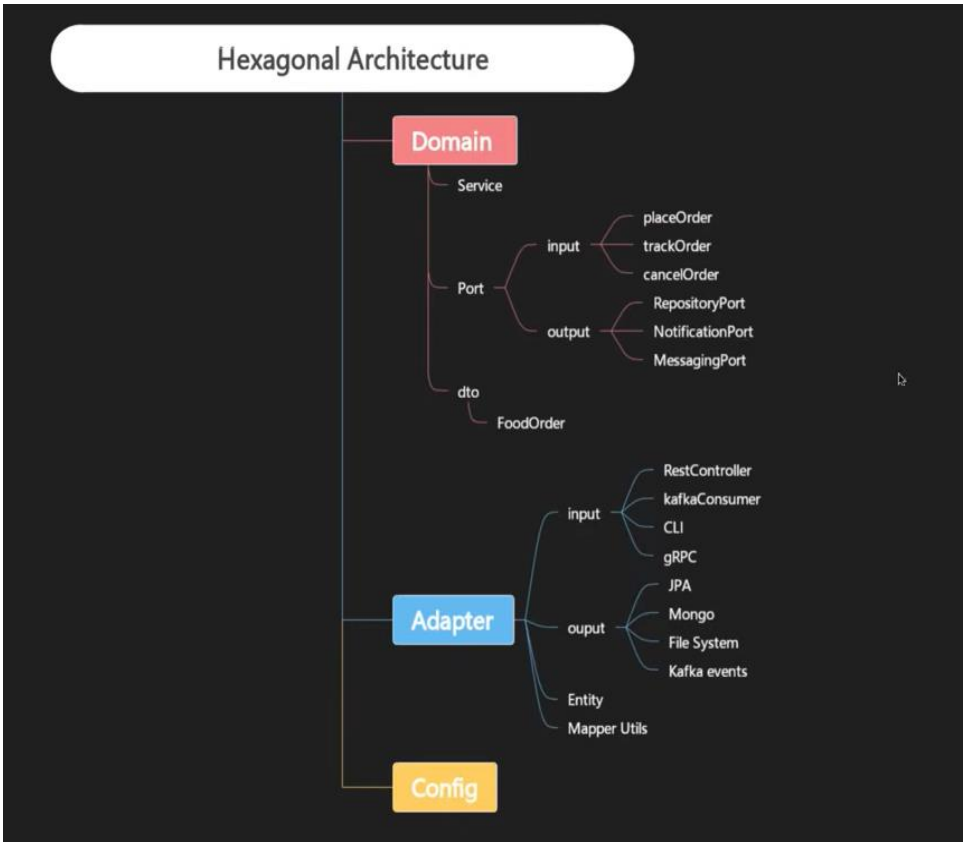


Example: Create Order Flow

Goal:

User creates an order → OrderService → DB We'll define:

- **InPort = CreateOrderUseCase interface**
- **OutPort = OrderRepositoryPort interface**



Spring Boot Code Snippet (Hexagonal Layering)

1. Domain (Core)

```
// InPort
public interface CreateOrderUseCase {
 void createOrder(Order order);
}
// OutPort
public interface OrderRepositoryPort {
```

```
 void save(Order order);
}
// Core Service
@Service
public class OrderService implements CreateOrderUseCase {
 private final OrderRepositoryPort repository;
 public OrderService(OrderRepositoryPort repository) {
 this.repository = repository;
 }
 public void createOrder(Order order) {
 // Business logic here
 repository.save(order);
 }
}
```

---

2. Input Adapter – REST Controller

```
@RestController
@RequestMapping("/orders")
public class OrderController {
 private final CreateOrderUseCase createOrder;
 public OrderController(CreateOrderUseCase createOrder) {
 this.createOrder = createOrder;
 }
 @PostMapping
 public void create(@RequestBody Order order) {
 createOrder.createOrder(order);
```

```
 }
}
```

---

3. Output Adapter – JPA Repository

```
@Repository
public class OrderJpaAdapter implements OrderRepositoryPort {
 private final SpringDataOrderRepository jpaRepo;
 public OrderJpaAdapter(SpringDataOrderRepository jpaRepo) {
 this.jpaRepo = jpaRepo;
 }
 public void save(Order order) {
 jpaRepo.save(order);
 }
}
```

Adapter Types You Can Plug In

Adapter Type	Use Case
REST Controller	Accept HTTP requests
Kafka Listener	Receive events
JPA Adapter	Store data in DB
Email Adapter	Send notifications
Test Adapter (Mock)	Unit testing without side effects

Summary Table

Concept	Meaning
Port	Interface between core and outer world
Adapter	Plug that connects to real systems
Core	Clean, testable business logic
Testability	Mocks can be plugged in easily
Flexibility	Replace DB, UI, message queue easily

One-Liner to Remember

"Hexagonal Architecture keeps your business logic clean and pluggable — like sockets and chargers."

RestTemplate – Manual HTTP Client

What Is It?

RestTemplate is a low-level way to make HTTP calls. You manually handle:

- **HTTP methods (GET, POST)**
- **URLs**
- **Serialization/deserialization**
- **Headers**

Think of it like a manual phone call: you dial the number, talk, and hang up.

Example

@Service

public class OrderService {

@Autowired

private RestTemplate restTemplate;

public PaymentResponse getPaymentDetails(String orderId) {

String url = "http://PAYMENT-SERVICE/payments/" + orderId;

return restTemplate.getForObject(url, PaymentResponse.class);

}

}

FeignClient – Declarative HTTP Client

What Is It?

`FeignClient` is a **declarative REST client** built on top of `RestTemplate`. You define

an **interface**, and Spring/Netflix Feign handles the rest.

> Think of it like **saving a contact in your phone**. Just call the name — no need to

remember numbers.

Example

1. Enable Feign

```java

@SpringBootApplication

@EnableFeignClients

public class OrderServiceApp { }

2. Define Feign Client Interface

@FeignClient(name = "PAYMENT-SERVICE")

public interface PaymentClient {

@GetMapping("/payments/{orderId}")
```

PaymentResponse getPayment(@PathVariable("orderId") String orderId);

}

3. Use in Service

@Service

public class OrderService {

@Autowired

private PaymentClient paymentClient;

public PaymentResponse getPaymentDetails(String orderId) {

return paymentClient.getPayment(orderId);

}

}

Asynchronous Calls with `@Async`

What Is `@Async`?

`@Async` is a Spring annotation that tells the method to **run in a separate background

thread**, so the caller **doesn't wait** for it to finish.

> Think of it like:

> "Fire-and-forget" or "Do this in the background."

Use Cases in Microservices

| Use Case | Why Async? |

| ---------- | ---------- |

| Send Email after placing order | Don't delay response to user |

| Call another microservice (non-blocking) | Improve responsiveness |

| Logging, auditing, analytics | Shouldn't slow main request |

| Cleanup tasks, image uploads, etc. | Background processing |

How It Works

When you call a method annotated with `@Async`, Spring:

* Creates a **separate thread**
* Executes the method **asynchronously**
* Main thread **continues immediately**

Step-by-Step Implementation

Step 1: Enable Async in Your App

```java
@SpringBootApplication
@EnableAsync
public class MyApp { }
```

Step 2: Add Async Method

```
@Service
public class EmailService {
 @Async
 public void sendEmail(String to, String subject, String body) {
 // simulate delay
 Thread.sleep(3000);
 System.out.println(" Email sent to " + to);
 }
}
```

This method will now run in the background.

Step 3: Call Async Method

@RestController

public class OrderController {

@Autowired

private EmailService emailService;

@PostMapping("/order")

public String placeOrder(@RequestBody OrderRequest order) {

// save order...

// async call

emailService.sendEmail(order.getEmail(), "Order Placed", "Thank you!");

return "Order Placed Successfully!";

}

}

Important Notes

| Rule | Description |

| ---------- | ---------- |

| Must enable with `@EnableAsync` | At config or main class |

| Method must be in a **separate class | `@Async` won't work if called from same class |**

| Return type can be `void`, `Future`, `CompletableFuture` | Choose based on need |

Optional: Customize Async Thread Pool

```java
@Configuration
@EnableAsync
public class AsyncConfig {
 @Bean(name = "taskExecutor")
 public Executor taskExecutor() {
 ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
 executor.setCorePoolSize(3);
 executor.setMaxPoolSize(10);
 executor.setQueueCapacity(100);
 executor.setThreadNamePrefix("async-task-");
 executor.initialize();
 return executor;
 }
}
```

Then use:

```java
@Async("taskExecutor")
public void someAsyncMethod() { ... }
```

Circuit Breaker Pattern

Simple Definition

A Circuit Breaker is like a safety switch that stops calling a failing service for a while, to avoid overloading it or causing cascading failures in your system.

Think of it like: “If your neighbor’s doorbell is broken, stop ringing it for some time instead of pressing it again and again.”

Problem It Solves

Without Circuit Breaker	With Circuit Breaker
Continues hitting failing service	Stops calls temporarily
Wastes threads and resources	Protects your service from crashing
Chain reaction of failures	Limits failure spread

Circuit States

State	What It Means
Closed	Everything is fine, calls go through
Open	Too many failures, stop calling the service
Half-Open	Test with limited calls to see if service is back

If test calls succeed → go back to Closed If they fail → back to Open

---

Real-World Use Case

- **You call a Payment Service from Order Service.**
- **Payment Service goes down.**
- **Without a circuit breaker → every request hangs or fails**
- **With circuit breaker → calls stop after N failures, and retry after a wait time**

Implementing Circuit Breaker in Spring Boot (Resilience4j)

Step 1: Add Dependency

<dependency>

```
<groupId>io.github.resilience4j</groupId>
<artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

Step 2: Annotate with @CircuitBreaker

```
@Service
public class PaymentService {
 @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
 public String callPaymentAPI() {
 // simulate external call
 return restTemplate.getForObject("http://payment-service/pay", String.class);
 }
 public String fallbackPayment(Exception e) {
 return "Payment service is currently unavailable. Try again later.";
 }
}
```

Step 3: Add Config (Optional - application.yml)

```
resilience4j:
 circuitbreaker:
 instances:
 paymentService:
 registerHealthIndicator: true
 slidingWindowSize: 5
 failureRateThreshold: 50
 waitDurationInOpenState: 10s
```

Feign + Circuit Breaker (Spring Cloud OpenFeign + Resilience4j)

```
@FeignClient(name = "payment-service", fallback = PaymentFallback.class)
public interface PaymentClient {
 @GetMapping("/pay")
 String pay();
}
@Component
public class PaymentFallback implements PaymentClient {
 public String pay() {
 return "Payment service is down. Please try later.";
 }
}
```

Key Config Terms

Config Name	Meaning
slidingWindowSize	How many calls to consider for failure rate
failureRateThreshold	% of failures to trip the breaker
waitDurationInOpenState	How long to wait before trying again

One-Liner to Remember

“A Circuit Breaker stops calling broken services, giving them time to recover — and keeps your system safe.”